

# BoxyHQ SaaS Starter Kit

Project Setup, Database Schema & Push Schema Guide

Version: 1.6.0

License: Apache 2.0

Generated: July 2026

# 1. How to Set Up the Project

## 1.1 Prerequisites

- **Node.js** v18+
- **npm** v9+
- **Docker Desktop** (for local PostgreSQL)
- **Git**

## 1.2 Step-by-Step Setup

### Step 1: Fork & Clone the Repository

```
git clone https://github.com/<your_username>/saas-starter-kit.git
cd saas-starter-kit
```

### Step 2: Install Dependencies

```
npm install
```

This installs all project dependencies including Next.js, Prisma, NextAuth.js, Stripe SDK, SAML Jackson, and more.

### Step 3: Configure Environment Variables

```
cp .env.example .env
```

Then edit the `.env` file. Key variables you **must** configure:

| Variable                     | Description                       | Default                                                                     |
|------------------------------|-----------------------------------|-----------------------------------------------------------------------------|
| <code>DATABASE_URL</code>    | PostgreSQL connection string      | <code>postgresql://postgres:postgres@localhost:5432/saas-starter-kit</code> |
| <code>NEXTAUTH_SECRET</code> | Encryption secret for NextAuth.js | (generate one)                                                              |

|                                                           |                                                   |                                    |
|-----------------------------------------------------------|---------------------------------------------------|------------------------------------|
| <code>NEXTAUTH_URL</code>                                 | Application URL                                   | <code>http://localhost:4002</code> |
| <code>APP_URL</code>                                      | Same as <code>NEXTAUTH_URL</code>                 | <code>http://localhost:4002</code> |
| <code>SMTP_HOST</code> ,<br><code>SMTP_PORT</code> , etc. | Email configuration for magic links & invitations | —                                  |

**Note:** Some features (Stripe, Svox, SAML SSO) require additional API keys. The app will work for core functionality without them, but those features will be disabled.

## Step 4: Start PostgreSQL (Docker)

```
docker-compose up -d
```

This starts a PostgreSQL 16.4 container on port 5432 with the following defaults:

- **Database:** `saas-starter-kit`
- **User:** `postgres`
- **Password:** `postgres`

## Step 5: Push the Database Schema

```
npx prisma db push
```

This synchronizes the Prisma schema from `prisma/schema.prisma` to your PostgreSQL database, creating all tables, indexes, and enums.

**What does this do?** See Chapter 3 for a full explanation of "push schema."

## Step 6: (Optional) Seed the Database

```
npx prisma db seed
```

Populates the database with sample data: 10 users, 5 teams, team memberships, and invitations.

## Step 7: Start the Development Server

```
npm run dev
```

The app will be available at **http://localhost:4002**.

## Step 8: (Optional) Open Prisma Studio

```
npx prisma studio
```

Opens a visual database browser at **http://localhost:5555**.

## 1.3 Available npm Scripts

| Command                          | Description                                                                                                   |
|----------------------------------|---------------------------------------------------------------------------------------------------------------|
| <code>npm run dev</code>         | Start development server (port 4002)                                                                          |
| <code>npm run build</code>       | Build for production ( <code>prisma generate</code> + <code>prisma db push</code> + <code>next build</code> ) |
| <code>npm start</code>           | Start production server (port 4002)                                                                           |
| <code>npm test</code>            | Run Jest unit tests                                                                                           |
| <code>npm run test:e2e</code>    | Run Playwright end-to-end tests                                                                               |
| <code>npm run sync-stripe</code> | Sync Stripe products & prices                                                                                 |

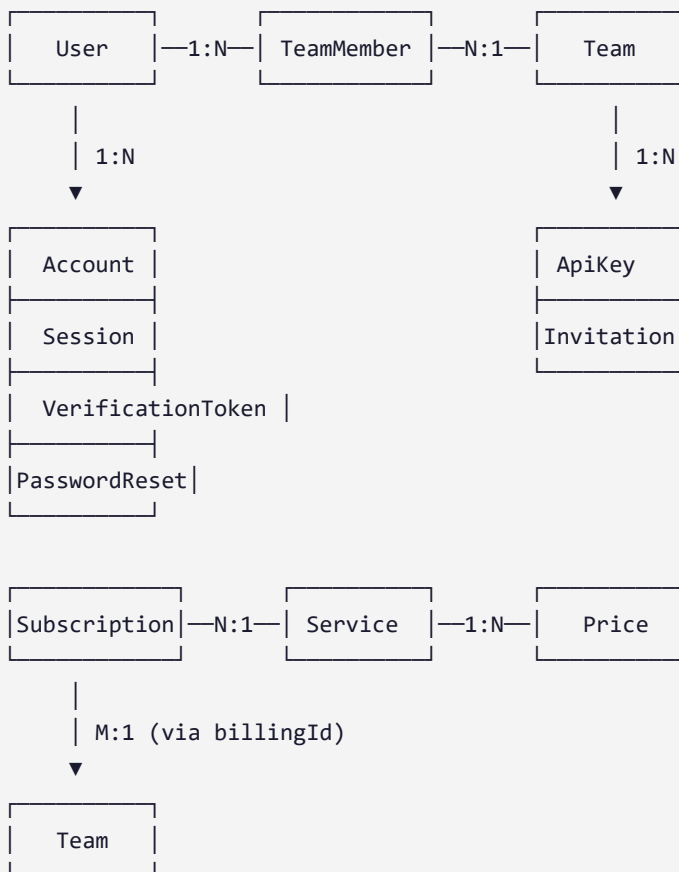
## 2. Database Schema

### 2.1 Overview

The project uses **Prisma ORM** with **PostgreSQL**. The schema is defined in `prisma/schema.prisma` and contains **12 models**:

- 8 application models (User, Team, TeamMember, Invitation, Account, Session, VerificationToken, PasswordReset, ApiKey, Subscription, Service, Price)
- 3 Jackson internal models (jackson\_store, jackson\_index, jackson\_ttl) — marked with `@@ignore`

### 2.2 Entity-Relationship Diagram (Textual)



### 2.3 Model Details

## Enum: Role

```
enum Role { ADMIN, OWNER, MEMBER }
```

## User

| Field                  | Type          | Constraints                            |
|------------------------|---------------|----------------------------------------|
| id                     | String (UUID) | Primary key, auto-generated            |
| name                   | String        | Required                               |
| email                  | String        | Unique, required                       |
| emailVerified          | DateTime?     | Nullable                               |
| password               | String?       | Nullable (null for social login users) |
| image                  | String?       | Nullable (avatar URL)                  |
| invalid_login_attempts | Int           | Default: 0                             |
| lockedAt               | DateTime?     | Nullable                               |
| createdAt              | DateTime      | Default: now()                         |
| updatedAt              | DateTime      | Auto-updated                           |

**Relations:** Has many TeamMembers, Accounts, Sessions, and Invitations.

## Account (NextAuth.js adapter)

Stores OAuth/SAML provider account links to a User. Key fields: `provider`, `providerAccountId`, `access_token`, `refresh_token`, etc.

## Session (NextAuth.js adapter)

Stores active user sessions. Linked to a User via `userId`.

## VerificationToken (NextAuth.js adapter)

Stores magic link verification tokens. Fields: `identifier`, `token`, `expires`.

## Team

| Field | Type          | Constraints |
|-------|---------------|-------------|
| id    | String (UUID) | Primary key |

|                             |         |                                    |
|-----------------------------|---------|------------------------------------|
| name                        | String  | Required                           |
| slug                        | String  | Unique, URL-friendly               |
| domain                      | String? | Unique, nullable (verified domain) |
| defaultRole                 | Role    | Default: MEMBER                    |
| billingId / billingProvider | String? | Stripe billing info                |

**Relations:** Has many TeamMembers (users), Invitations, and ApiKeys.

## TeamMember

Junction table between User and Team. Unique constraint on (teamId, userId). Each member has a Role.

## Invitation

| Field          | Type     | Description         |
|----------------|----------|---------------------|
| token          | String   | Unique invite token |
| email          | String   | Invited email       |
| expires        | DateTime | Expiry timestamp    |
| invitedBy      | String   | References User     |
| sentViaEmail   | Boolean  | Default: true       |
| allowedDomains | String[] | Domain allow-list   |

Unique constraint on (teamId, email).

## PasswordReset

Stores password reset tokens. Uses auto-increment integer primary key.

## ApiKey

| Field     | Type      | Description            |
|-----------|-----------|------------------------|
| hashedKey | String    | Unique, hashed API key |
| teamId    | String    | Foreign key to Team    |
| expiresAt | DateTime? | Nullable               |

|            |           |          |
|------------|-----------|----------|
| lastUsedAt | DateTime? | Nullable |
|------------|-----------|----------|

### Subscription

Stores Stripe subscription data. Fields: `customerId`, `priceId`, `active`, `startDate`, `endDate`.

### Service & Price

Billing catalog: **Service** has many **Prices**. Used for Stripe product/price sync.

## 2.4 Migrations

The project includes **11 migration files** in `prisma/migrations/`:

| Migration                                               | Date     | Changes                                                         |
|---------------------------------------------------------|----------|-----------------------------------------------------------------|
| <code>20230625203909_init</code>                        | Jun 2023 | Initial schema (User, Team, TeamMember, Account, Session, etc.) |
| <code>20230703084002_add_api_keys</code>                | Jul 2023 | Added ApiKey model                                              |
| <code>20230703084117_add_index_user</code>              | Jul 2023 | Index on User                                                   |
| <code>20230720113226_make_password_optional</code>      | Jul 2023 | Made password nullable                                          |
| <code>20231109082527_add_account_lockout</code>         | Nov 2023 | Added account lockout fields                                    |
| <code>20240109041326_columns_for_invitation_link</code> | Jan 2024 | Added sentViaEmail, allowedDomains                              |
| <code>20240212105842_stripe</code>                      | Feb 2024 | Subscription, Service, Price models                             |
| <code>20240213160517_indices</code>                     | Feb 2024 | Various indexes                                                 |
| <code>20240226091046_add_jackson_schema</code>          | Feb 2024 | Jackson internal models                                         |
| <code>20240316224800_add_index_billing_id</code>        | Mar 2024 | Index on Team.billingId                                         |



|                                                |          |                           |
|------------------------------------------------|----------|---------------------------|
| 20240531124127_expand_jackson_namespace_column | May 2024 | Expanded namespace column |
|------------------------------------------------|----------|---------------------------|

## 3. What Is "Push Schema" and Why Is It Needed?

---

### 3.1 Definition

---

"Push schema" refers to the Prisma CLI command:

```
npx prisma db push
```

This command synchronizes your Prisma schema ( `prisma/schema.prisma` ) directly to your database — creating, altering, or deleting tables, columns, indexes, and enums to match the schema definition.

### 3.2 How It Works

---

1. **Reads** the `schema.prisma` file (the source of truth for your database structure)
2. **Connects** to the database using `DATABASE_URL` from `.env`
3. **Compares** the schema against the current database state
4. **Generates** and executes SQL statements to make the database match the schema
5. **Reports** any warnings (e.g., data loss if a column is removed)

### 3.3 Why Is It Needed?

---

#### 1. First-Time Database Setup

After cloning the project and starting PostgreSQL, the database is empty. Running `prisma db push` creates all the tables, columns, indexes, and enums defined in the schema so the application can function.

**Without this step:** The app would crash with "relation does not exist" errors when trying to query users, teams, etc.

#### 2. Schema Changes During Development

When you modify `schema.prisma` (add a field, change a type, add a model), `prisma db push` applies those changes immediately without creating migration files. This is ideal for rapid prototyping.

#### 3. Keeping Prisma Client Updated

After pushing schema changes, you also need to regenerate the Prisma Client:

```
npx prisma generate
```

This updates the TypeScript client library with type-safe methods for the new/changed models.

### 3.4 Push Schema vs. Migrations

| Aspect             | <code>prisma db push</code> | <code>prisma migrate</code>                 |
|--------------------|-----------------------------|---------------------------------------------|
| Purpose            | Quick sync for development  | Version-controlled schema changes           |
| Migration files    | Does NOT create any         | Creates SQL migration files                 |
| Version history    | No history                  | Full change history                         |
| Rollback           | Not supported               | Supported via migration files               |
| Data loss warnings | Interactive prompts         | Checked during generation                   |
| Production use     | Not recommended             | Recommended ( <code>migrate deploy</code> ) |
| Speed              | Fast (no file generation)   | Slower (generates files)                    |
| Team collaboration | Harder (no file to commit)  | Easier (files in version control)           |

**In this project:** The `package.json` `build` script runs `prisma db push` automatically: `prisma generate && prisma db push && next build`. This ensures the database schema is always up-to-date before building.

### 3.5 When to Use Each

- **Use `prisma db push`:** During initial development, prototyping, or when you're the only developer working on the database schema.
- **Use `prisma migrate dev`:** When you want to create a formal migration file to commit to version control, especially in a team setting.
- **Use `prisma migrate deploy`:** In production/staging environments to apply migrations safely.

### 3.6 Common Commands Cheat Sheet

| Command                         | What It Does                           |
|---------------------------------|----------------------------------------|
| <code>npx prisma db push</code> | Push schema to DB (no migration files) |

|                                        |                                       |
|----------------------------------------|---------------------------------------|
| <code>npx prisma generate</code>       | Regenerate Prisma Client (TypeScript) |
| <code>npx prisma studio</code>         | Open visual database browser          |
| <code>npx prisma migrate dev</code>    | Create and apply a new migration      |
| <code>npx prisma migrate deploy</code> | Apply pending migrations (production) |
| <code>npx prisma db seed</code>        | Seed database with sample data        |
| <code>npx prisma db pull</code>        | Introspect DB and generate schema     |

## 3.7 Summary

**Push schema ( `prisma db push` ) is needed because:**

1. The database starts empty — tables must be created for the app to work.
2. It provides a fast, iterative workflow during development.
3. It keeps the database in sync with the Prisma schema as you make changes.
4. It's integrated into the build pipeline (see `package.json` : `"build": "prisma generate && prisma db push && next build"` ).

**Pro Tip:** For team projects, consider switching to `prisma migrate dev` to generate migration files that can be reviewed in pull requests and applied consistently across environments.